

addition

We add binary numbers one column at a time, from the right, just as in decimal. A column can add up to at most three, so it always gives one bit to write and at most one to carry.

$$0 + 0 = 0 \quad \text{write 0}$$

$$0 + 1 = 1 \quad \text{write 1}$$

$$1 + 1 = 10 \quad \text{write 0, carry 1}$$

$$1 + 1 + 1 = 11 \quad \text{write 1, carry 1}$$

the third 1 is a carry coming in from the column to the right

To add 0101 (5) and 0011 (3), we work right to left, writing each column's carry above the next.

	8	4	2	1	
	1	1	1		
	0	1	0	1	5
+	0	0	1	1	3
	1	0	0	0	= 8

each 1 + 1 writes 0 and carries 1 into the next column

With n bits, a sum can need $n + 1$. The extra bit is the **carry out**; the result keeps only n bits, so the carry out is lost.

		8	4	2	1	
	1					
		1	0	0	1	9
+		1	1	0	0	12
	1	0	1	0	1	= 5, not 21

1 carry out

for unsigned numbers, a carry out of 1 means the sum did not fit

Two's-complement numbers add in exactly the same way. The carry out is still dropped, but here the answer is right.

		-8	4	2	1	
	1	1				
		1	1	0	1	-3
+		1	1	1	0	-2
	1	1	0	1	1	= -5

the same adder serves both readings; only the check for a wrong answer differs

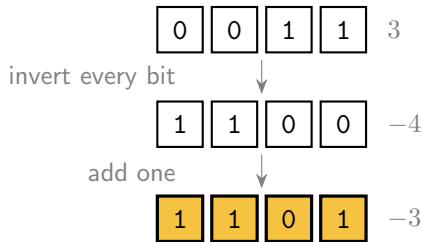
When two numbers of the same sign give a result of the other sign, the answer has **overflowed**.

		-8	4	2	1	
	0	1				
		0	1	0	1	5
+		0	1	0	0	4
		1	0	0	1	= -7, not 9

the carry into the sign bit (1) differs from the carry out of it (0)

subtraction

We subtract by adding the negation: $a - b = a + (-b)$. To negate b , we invert every bit, then add one.



then an ordinary addition finishes the job

To compute $0101 - 0011$ ($5 - 3$), we add 0101 and $-3 = 1101$.

		8	4	2	1	
	1	1		1		
		0	1	0	1	5
+		1	1	0	1	-3
	1	0	0	1	0	= 2

the carry out is dropped, as in any addition

For unsigned numbers, the carry out now tells us whether we had to **borrow**: it is 1 when $a \geq b$ and 0 when $a < b$.

$$\begin{array}{r}
 \begin{array}{|c|c|c|c|} \hline 0 & 1 & 1 & 1 \\ \hline \end{array} 7 \\
 + \begin{array}{|c|c|c|c|} \hline 1 & 0 & 1 & 1 \\ \hline \end{array} -5 \\
 \hline
 \begin{array}{|c|c|c|c|} \hline 1 & 0 & 0 & 1 & 0 \\ \hline \end{array} = 2
 \end{array}$$

carry out 1:
no borrow, correct

$$\begin{array}{r}
 \begin{array}{|c|c|c|c|} \hline 0 & 1 & 0 & 1 \\ \hline \end{array} 5 \\
 + \begin{array}{|c|c|c|c|} \hline 1 & 0 & 0 & 1 \\ \hline \end{array} -7 \\
 \hline
 \begin{array}{|c|c|c|c|} \hline 0 & 1 & 1 & 1 & 0 \\ \hline \end{array} = 14
 \end{array}$$

carry out 0:
a borrow, not -2

Signed subtraction overflows when a and b have different signs and the result's sign isn't a 's.

		-8	4	2	1	
	1					
	0	1	0	1		5
+	0	1	0	0		$-(-4) = 4$
	1	0	0	1		$= -7, \text{ not } 9$

it overflows exactly when the addition $a + (-b)$ does

The same adder does all four, with $a - b$ computed as $a + \sim b + 1$. Only our reading of the bits changes, and with it how we tell the answer is **wrong**.

	wrong when	example (4 bits)
unsigned $a + b$	$\text{cout} = 1$	$9 + 12 \rightarrow 0101 (5)$
unsigned $a - b$	$\text{cout} = 0$ (a borrow)	$5 - 7 \rightarrow 1110 (14)$
signed $a + b$	inputs share a sign, the result has the other	$5 + 4 \rightarrow 1001 (-7)$
signed $a - b$	inputs differ in sign, the result's sign isn't a's	$5 - (-4) \rightarrow 1001 (-7)$

unsigned checks carry; signed checks overflow ($\text{cin} \neq \text{cout}$ at the sign bit)